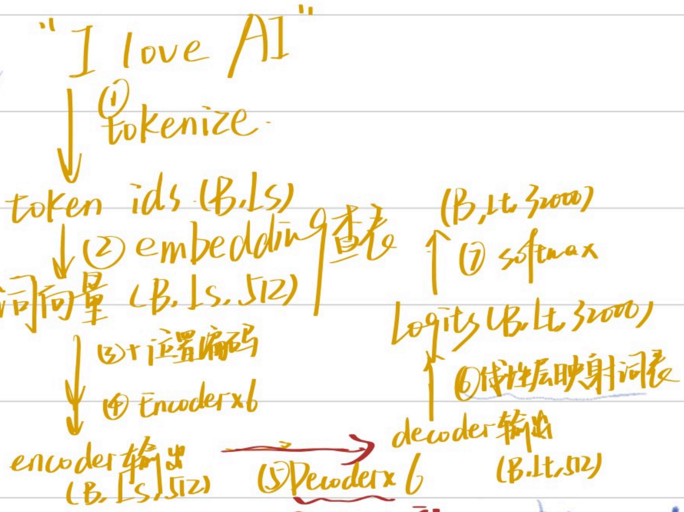
Transformer数据流与张量形状追踪

(B, batch size 一次送几个句子, d-model 模型隐藏维度, h. 注意力头数, d-k = d-v 每个头的维度, d-ff 前馈网络的中间维度)

Q: "我在找什么"
K: "我是什么"
V: "我的内容"

全局数据流


"I love AI"
① tokenize
token ids (B,Ls)
② embedding 查表
词向量 (B,Ls,512)
③ + 位置编码
④ Encoder×6
encoder输出 (B,Ls,512)
⑤ Decoder×6
decoder输出 (B,Lt,512)
⑥ 线性层映射词表
Logits (B,Lt,32000)
⑦ softmax
(B,Lt,32000)

交叉注意力 (Q来自Decoder, K,V来自Encoder)

{Encoder内部}

step1. 三次投影 Q=x@W-Q, W-Q(512,512), 输出(B,Ls,512),

K,V同理 step2. 一劈形成多头 (reshape+transpose) (B,Ls,512) → 切成8份 → (B,Ls,8,64) → 转 → (B,8,Ls,64), 含义: (Batch, 头, 序列位置, 每头维度) step3. 计算注意力分数, Q(B,8,Ls,64) @K^T(B,8,64,Ls) → (B,8,Ls,Ls). 别忘除 $\sqrt{dk}$. step4. softmax 归一

step5. 加权求和取value, out=weights @V, (B,8,Ls,Ls)@(B,8,Ls,64) → (B,8,Ls,64) step6. 合并多头 (transpose + reshape) step7. out=out@W-O (512,512) → (B,Ls,512)

①

Add&Norm (残差+层归一化) step1. X=LayerNorm(x+Attention(x))

残差: x+Attention输出, 逐元素加, 形状不变, LayerNorm: 沿最后一维(512)做归一化(减均值除标准差)

②

前馈网络 FFN (每个位置独立加工), FFN(x)=Relu(x@W1+b1)@W2+b2.

③

Decoder单层内部 (此处只说difference)

masked自注意力, 交叉注意力 (cross-Attention)

GPT内部结构（Decoder-only Transformer）

输入 token IDs

↓

Token Embedding + position Embedding

↓

Decoder Block × N

$x = x + \text{Masked Self Attention}(\text{LayerNorm}(x))$

$x = x + \text{FFN}(\text{LayerNorm}(x))$

↓

LayerNorm

↓

LM Head

↓

下一个token概率

训练时

Decoder Block:

（详细内部结构）

x

↓

LayerNorm

↓

Masked Self-Attention

↓ Q = x@W_Q.

K = x@W_K

V = x@W_V

↓ 拆多头

↓ scores = Q@K^T/sqrt(d_head)

多头的维度.

加mask.

softmax.

↓ out = weights@V.

↓ 合并多头

↓ out = out@W_O.

x = x + out ← 残差.

↓

LayerNorm

↓

FFN

h = x@W1 + b1

↓ h = GELU(h)

out = h@W2 + b2

x = x + out ← 残差 2.

↓

输出 x

Prefill + Decode.

推理时

(1) Decoder block.

① prefill

token IDs[1,5] —Embedding / Position→ X[1,5,D] —(2) Masked Self-attn (3) FFN→ X:[1,5,D] ↓ LM Head Logits:[1,5,V]

输入 → 模型前向 → Logits —取最后位置→ Last_logits[1,100]

↓ Softmax

probs:[1,100] —采样→ next_token → 拼回输入

② Decode

算新token. Q,K,V —拆多头→ 拼到KV cache ↓ 算注意力 → 加mask → 加权求和 → 合并头 输出投影 → 残差 FFN → LM Head ↓ 重复

out= weights @ V_cache

key:在每次算完k,v后将k,v存起来

Torch.arrange 创建张量

device = idx.device 自动获取输入数据所在设备

if instance( ) 识别层类型，统一初始化

assert 一堆断言，防止最后报错，提前终止，(类C).

view：展平 ignore_index=-1(忽略与矩阵运算时添加的(-1)), pad;

(添的位置)

进行展平后，计算交叉熵，而Batch与len维度展开，刚好能一一对应，故而可进一步算交叉熵

GPT构型，只追溯张量

① 输入 (B,L), (int64)

② 查表 + 位置矩阵 (B,L,768) (float32)

(wte:(50257,768)

③ Pre-Layernorm + 因果自注意力

h = Layernorm(x)

Q = h @ W_Q (768,768) → (B,L,768)

同理，K,V也是768

④ 分头 (B,L,768) → reshape(B,L,12,64)

↓ transpose

(B,12,L,64)

$scores = Q @ K^T / \sqrt{64}$ (B,12,L,L)

⑤ softmax，只有最后一维 (B,12,L,L)

⑥ out=weights @ V (B,12,L,64)

合并 transpose + reshape → (B,L,768)

@W_O (768,768) → (B,L,768)

汇聚不同头的信息，去除噪声.

Attention dropout → (B,L,768)

↓ (得到上下文)

⑦ 残差

x = x + attn_out → (B,L,768)

⑧ pre-Layernorm + FFN

h = Layernorm(x) → (B,L,768)

h = h @ W1, W1(768,3072) → (B,L,3072)

h = Gelu(h)

h = h @ W2, W2(3072,768) (B,L,768)

X = X + h → (B,L,768)

<精炼内容>

⑨ Final Layernorm (B,L,768)

LM_Head (@W_lm (768,50257) (B,L,50257)

⑩ softmax 最后一维

crossEntropy

elif = else if

```
for pn, p in self.named_parameters():
    if pn.endwith('c_proj.weight'):
        torch.nn.init.normal_(p, mean=0.0, std=0.02/math.sqrt
        (2*config.n_layer))
```

合理缩放初始化的残差
防止n_layer过多,使得累加量过多 ←

```
for block in self.transformer.h:
    x = block(x)
x = self.transformer.ln_f(x)
```

(final layernorm)

☆ 一串

→ 前期层层嵌套,现在直接block串起来

---

logits = logits[:, -1, :] / temperature 取最后一个位置的logits,除以温度

```
idx_next = torch.multinomial(probs, num_samples=1)
```

→ 按概率采样

☆☆ [:, -1, :]

↓ 维度0 ↓ 维度1 ↓ 维度2

"-1" → 只取最后一个 token

":" 全部维度都取

":256" 只留前256维

补: mask_buffer (下三角矩阵)是固定的、在参数搬运的时候,自己会生成,无需从外面搬。

---

hasattr(对象, "属性名") = "这个对象有没有这个属性?"

有 → true　　没有 → False

代码示例: if hasattr(block.attn, 'bias'):

优化器选择: 参数分两组, 权重矩阵+weight decay, bias/LayerNorm不加